ForgeData™

Geospatial Data Routing, Native to MCP

An MCP server an agent hands a data question to — which dataset covers this area at the right resolution, and what does it cost to read? — and gets back a ranked, costed answer instead of a hardcoded path, with structured errors it can branch on. ForgeData™ keeps one catalog across fourteen source types and routes every request fidelity-first, so the agent reasons about intent and never about where a file lives.

Intent → data

name the operation, get the dataset

find_optimal_for_operation returns ranked, deduplicated candidates for an operation and an area

Typed

errors an agent can branch on

every refusal carries an error code, and since 1.2.6 every reply with no data names an empty_reason

Bounded

replies sized for a context window

lists page by default; large geometry goes to a file instead of the reply

Ask by intent

Name an operation — slope, viewshed, NDVI — and an area, and `find_optimal_for_operation` ranks what can serve it: locally available derived products first, then raw data by adequate coverage, finest resolution and extent-fit, with access tier only breaking the tie, deduplicated to a minimal covering set. Candidates carry an id; `get_dataset` resolves it. `estimate_access_cost` explains any single choice.

Graceful with sparse metadata

Thin or unknown metadata gives a coarser ranking, not an empty result: unknown resolution is kept unless you set a hard floor, unknown band counts pass, and single-band imagery still routes. When nothing covers the area, the reply names why and which registered source could acquire it — the next tool call, spelled out.

Memory the agent can use

An agent registers what it computes as a derived product — operation, parameters, and its parent when it is in the same catalog — and the next routing call prefers it over recomputing. `publish_file` and `publish_vector` put an agent's output into the catalog in one idempotent call, deduplicated by content hash and immediately searchable.

A versioned contract (1.2.6). The tool surface ships as a contract snapshot inside the jar, reported by a hidden `contract_info` tool. 1.2.6 is contract 2.0.0; its one breaking change is that `read_raster_window` returns the file's real data type — an INT16 extract stays INT16 — instead of always 32-bit float.

Geodesic distances (1.2.6). Profile distances, road lengths and distances to a road are measured on the WGS84 ellipsoid through ForgeGIS's measurement core, so an agent's numbers match ForgeGIS Studio's Measure of the same line.

CONTINUED ON REVERSE →

WHO THIS IS FOR

Teams building on ForgeMind™

ForgeMind runs ForgeData as a stdio child and asks it for data on every turn that needs some; Seaglass Globe™ runs its own copy for its catalog panel, and ForgeGIS Studio™ embeds it as a library. All of them read the same catalogs, so an answer found in one window is findable in the others.

MCP-native agent platforms

Any MCP client can spawn ForgeData over stdio and use the same surface: catalog search and detail, elevation at points, along profiles and inside polygons, windowed raster extracts, vector features and road lookup, routing, publishing, derived products, source management and sync. Local paths come back absolute, so a compute engine can open them directly.

Builders of long-running workflows

Federated catalogs merge several buckets behind namespaced ids, remote access costs calibrate themselves on measured latency and persist across restarts, and a temporal filter narrows data to a date range. The longer a project runs, the more of what it needs is already in the catalog.

Answers that carry their own caveats (1.2.6). Elevation replies carry `vertical_units`, `vertical_datum` and `vertical_source`; values are never converted, so a feet DEM answers in feet and says so. Region statistics add `coverage_fraction` (0–1) and `tiles_skipped`; a profile's `null` samples carry `sample_reasons` — `no_tile`, `water_mask`, `void` or `unreadable`.

Bounded replies, by design. `list_datasets` and the `search_by_*` tools return 25 rows by default with `total`, `truncated` and `next_offset`; `limit: 0` returns everything. Geometry tools stop inlining above 10 000 vertices; pass an `output_path` and the full result is written there, confined to the export root.

Errors worth branching on. `aoi_out_of_coverage`, `bbox_invalid`, `source_file_missing`, `rate_limited`, `timeout`, `operation_not_supported`, `catalog_read_only`, and since 1.2.6 `vector_crs_not_lonlat` and `point_cloud_units_unknown`; tools add their own, such as `aoi_too_large` with the requested and maximum areas. Empty replies name why: `no_aoi_coverage`, `covered_no_cells`.

Build on ForgeData

The Technical Brief covers the catalog, the cost model, the operation registry, every source type and the tool surface; the full tool reference ships with the suite documentation. ForgeData is bundled with the Forge suite and is not sold separately. The Forge suite is commercially licensed; evaluation builds are available to qualified teams.

rich@seaglassfoundry.com · seaglassfoundry.com

ForgeData™, ForgeGIS™, ForgeMind™, ForgeGIS Studio™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. GeoPackage, SQLite, Java, and the Model Context Protocol are marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.